

RnD Project Report

Call Feedback App

Gaurav Kumar
Shiv Narang

February 2026

1 Introduction

We are developing an app to record user feedback after a call ends. The app will record both objective feedback (like rating call quality) and subjective feedback (like comments). We will also record some system-measured quantities like signal strength, cellular generation, and location.

2 Tech Stack

Our frontend development was in Kotlin language, the most popular Android App development language. It had a wide support of different kind of APIs, which was ideal for our task. We developed this on Android Studio IDE. Since our backend tasks were simple, we used FastAPI for backend, and MongoDB as our database.

3 App Workflow

The app runs as a background service, and detects the call using TelephonyManager API. When it detects that call state is idle, after being in call, it pops-up an overlay to capture user feedback. Also, it captures device statistics like cellular generation, signal strength and location in the background.

When the user clicks submit all this data is sent to the backend via a POST request, and the backend adds it into its database.

4 Feedback

4.1 User Feedback

The overlay with user feedback has the following:

- **Voice Quality** Rating stars from 1-5

- **Audio Issues** Multiple selectable options including Call Dropped, Can't Hear them, They can't hear, background noise, echo.
- **Environment** Single selectable from Indoor, Outdoor, In Vehicle, Noisy Area.
- **Comments** Additional subjective feedback by the user

All these fields are optional with default null values.

4.2 Device Reported Data

We try to record the following from device:

- **Network Generation** Check if user is on WiFi or Cellular, and if on cellular, report generation also.
- **Signal Strength** Check the signal strength in dBm. This measurement is sometimes blocked by the device due to security reasons.
- **Location** We also try to fetch the last known exact location (or approximate, if exact not available).

5 APIs Used

This section describes the Android platform APIs used to collect network, radio, and contextual metadata for post-call Quality of Experience (QoE) analysis.

5.1 TelephonyManager

The **TelephonyManager** API provides access to cellular network and radio-related information. It is used to determine the current cellular network type and signal strength.

5.1.1 Call State Detection

Call state detection is performed using the **TelephonyManager** API to monitor changes in the device's telephony call state. The Android system reports call state transitions via a listener mechanism, allowing applications to react to call lifecycle events in real time.

The primary call states are represented by integer constants defined in **TelephonyManager**:

- **CALL_STATE_IDLE**: Indicates that the device is not currently involved in a call.
- **CALL_STATE_RINGING**: Indicates an incoming call is ringing.

- **CALL_STATE_OFFHOOK**: Indicates that a call is active, including outgoing calls or answered incoming calls.

Call state updates are received by registering a **PhoneStateListener** and overriding the **onCallStateChanged()** callback. Transitions between these states allow the application to infer call start and call end events. Specifically, a transition to **CALL_STATE_OFFHOOK** is interpreted as the start of a call, while a transition back to **CALL_STATE_IDLE** signifies the termination of a call.

This mechanism enables post-call actions such as triggering a user feedback interface or logging call-related metadata for network performance analysis.

5.1.2 Network Type Detection

The API method **getDataNetworkType()** returns an integer constant representing the current radio access technology. These values are mapped to high-level network generations as follows:

- **2G**: GPRS, EDGE, CDMA, 1xRTT, IDEN
- **3G**: UMTS, HSDPA, HSUPA, HSPA, EVDO variants
- **4G/LTE**: LTE
- **5G/NR**: NR

5.1.3 Signal Strength Estimation

The **allCellInfo** API is used to retrieve a list of available cell information objects. Depending on the radio technology, the signal strength is extracted from the corresponding classes:

- **CellInfoGsm**
- **CellInfoWcdma**
- **CellInfoLte**
- **CellInfoNr** (Android 12+)

The received signal strength indicator (RSSI), expressed in dBm, is extracted from the **CellSignalStrength** object. Invalid or unavailable values are filtered before use.

5.2 ConnectivityManager

The **ConnectivityManager** API is used to determine the currently active network transport.

Using **getActiveNetwork()** and **getNetworkCapabilities()**, the application identifies whether the device is connected via:

- **Wi-Fi** (`TRANSPORT_WIFI`)
- **Cellular** (`TRANSPORT_CELLULAR`)

This allows the system to distinguish between Wi-Fi-based calling and cellular-based calling scenarios.

5.3 FusedLocationProviderClient

The `FusedLocationProviderClient` API from Google Play Services is used to obtain the device’s approximate geographical location at the time of feedback submission.

The method `getCurrentLocation()` is invoked with `PRIORITY_BALANCED_POWER_ACCURACY`, which provides a coarse location estimate while minimizing battery consumption. Location access is performed only when the appropriate runtime permissions are granted.

6 Backend

The backend component of the system is responsible for receiving, validating, storing, and exposing call feedback and associated network performance metrics collected from the Android client.

6.1 Architecture

The backend follows a client–server architecture, where the Android application communicates with the server over a RESTful HTTP interface. Data is transmitted in JSON format to ensure platform independence and ease of extensibility.

6.2 Data Schema

The backend data model is defined using `Pydantic` schemas, which provide strong runtime validation, type safety, and automatic request parsing for incoming API payloads. The schema is designed to capture both subjective user feedback and objective device-level network metadata in a single unified record.

6.2.1 Enumerations

To ensure consistency and limit ambiguity in categorical inputs, enumerated types are used for commonly reported call conditions.

AudioIssueEnum This enumeration captures specific audio-related issues experienced during a call:

- **CALL_DROPPED**: The call was unexpectedly disconnected.

- **COULD_NOT_HEAR_OTHER**: The user could not hear the remote party.
- **OTHER_COULD_NOT_HEAR_ME**: The remote party could not hear the user.
- **BACKGROUND_NOISE**: Excessive background noise affected call clarity.
- **ECHO**: Echo was present during the call.

EnvironmentEnum This enumeration represents the physical environment in which the call was made:

- **INDOOR**: Call made indoors.
- **OUTDOOR**: Call made outdoors.
- **IN_VEHICLE**: Call made while traveling in a vehicle.
- **NOISY_AREA**: Call made in a noisy surrounding.

6.2.2 UserFeedback Schema

The **UserFeedback** schema represents a single feedback submission sent from the client application. All fields are optional to minimize user friction and allow partial submissions.

- **voiceQuality**: Integer rating from 1 to 5 representing perceived voice quality.
- **audioIssues**: List of reported audio issues selected from **AudioIssueEnum**.
- **environment**: Environment category selected from **EnvironmentEnum**.
- **comment**: Optional free-text comment provided by the user.

In addition to user-provided feedback, the schema embeds device-collected metadata:

- **networkGeneration**: Network type at the time of the call (e.g., WiFi, 4G, 5G).
- **signalStrength**: Cellular signal strength measured in dBm.
- **latitude, longitude**: Approximate geographic location of the device.
- **timestamp**: Unix timestamp (in milliseconds) indicating when the feedback was recorded.

6.2.3 Persistent Storage Schema

The **FeedbackInDB** schema extends **UserFeedback** by adding backend-managed fields:

- **id**: Unique identifier assigned to each feedback entry.
- **created_at**: Server-side timestamp indicating when the record was stored.

This separation ensures a clear distinction between client-provided data and backend-generated metadata.

6.3 Data Ingestion API

A dedicated endpoint is provided to ingest call feedback records generated after call termination. Each request contains anonymized metadata related to the call session, along with user-provided quality ratings.

To ensure data integrity, the backend performs schema validation and basic sanity checks before persisting the data.

6.4 Data Storage

Collected feedback is stored in a structured database optimized for analytical queries. Each feedback entry is stored as a single record, allowing efficient aggregation across dimensions such as network type, signal strength, time, and geographic region.

6.5 Security and Privacy

All communication between the client and backend is encrypted using HTTPS to prevent eavesdropping and tampering. The backend enforces strict access control for administrative endpoints and limits data exposure to aggregated or anonymized views.

User privacy is a primary design consideration. No phone numbers, contact identifiers, or call content are collected.